

Java Polymorphism (Overloading & Overriding) Practical Solutions

Note

These are merely proposed solutions. You may provide other and even better ones!

Exercise 1: Implementing Method Overriding

```
class Vehicle {
    protected String model;
    protected double rentalRate;

    public Vehicle(String model, double rentalRate) {
        this.model = model;
        this.rentalRate = rentalRate;
    }

    public void displayDetails() {
        System.out.println("Vehicle Model: " + model +
            ", Rental Rate: $" + rentalRate + " per day");
    }

    public double calculateRentalCost(int days) {
        return rentalRate * days;
    }
}

class Car extends Vehicle {
    private int seatingCapacity;

    public Car(String model, double rentalRate, int seatingCapacity) {
        super(model, rentalRate);
        this.seatingCapacity = seatingCapacity;
    }

    @Override
    public void displayDetails() {
        super.displayDetails();
        System.out.println("Seating Capacity: " + seatingCapacity);
    }
}

class Bike extends Vehicle {
```

```

    public Bike(String model, double rentalRate) {
        super(model, rentalRate);
    }

    @Override
    public double calculateRentalCost(int days) {
        return (rentalRate * days) * 0.9; // 10% discount for bikes
    }
}

```

Exercise 2: Implementing Method Overloading

```

class RentalService {

    public void rentVehicle(Vehicle vehicle, int days) {
        System.out.println("Renting a vehicle for " + days +
            " days. Total cost: $" +
            vehicle.calculateRentalCost(days));
    }

    public void rentVehicle(Vehicle vehicle) {
        rentVehicle(vehicle, 1); // Default rental for one day
    }
}

```

Exercise 3: Combining Overloading and Overriding

```

class Vehicle {

    public double calculateRentalCost(int days, boolean insurance) {
        double total = rentalRate * days;
        if (insurance) {
            total += 10 * days;
        }
        return total;
    }
}

class Bike extends Vehicle {

```

```

@Override
public double calculateRentalCost(int days, boolean insurance) {
    double total = (rentalRate * days) * 0.85; // 15% discount
    if (insurance) {
        total += 10 * days;
    }
    return total;
}
}

```

Exercise 4: Polymorphic Rental System

```

abstract class Transport {

    String model;

    public Transport(String model) {
        this.model = model;
    }

    abstract void move();

    public void showModel() {
        System.out.println("Model: " + model);
    }
}

class Car extends Transport {

    public Car(String model) {
        super(model);
    }

    @Override
    void move() {
        System.out.println(model + " is driving.");
    }
}

class Bike extends Transport {

    public Bike(String model) {
        super(model);
    }
}

```

```
@Override
void move() {
    System.out.println(model + " is riding.");
}
}
```

Exercise 5: Advanced Rental Service

```
interface LuxuryService {
    void offerLuxuryPackage();
}
```

```
class Car extends Vehicle implements LuxuryService {
```

```
    public Car(String model, double rentalRate) {
        super(model, rentalRate);
    }
```

```
@Override
    public void offerLuxuryPackage() {
        System.out.println(model + " includes a luxury package: " +
            "leather seats, free WiFi, and a chauffeur.");
    }
}
```